

PROYECTO FINAL ARQUITECTURA DE SOFTWARE
LAVADERO PULIDO ROJO

Programa: Ingeniería De Sistemas
Asignatura: Arquitectura De Software
Profesor: Jesús Ariel Gonzales

Integrantes
María Sofía Aljure Herrera
Brayan Smith Bedoya
David Felipe Perdomo Castillo

Corporación Universitaria Del Huila Corhuila
Neiva – Huila
Noviembre 2025

Introducción

El presente proyecto tiene como finalidad diseñar y estructurar la arquitectura de software para un sistema de gestión de un lavadero de carros, incluyendo los módulos de vehículos, usuarios, servicios y reservación. Se utilizará una arquitectura basada en microservicios con enfoque hexagonal, lo que permitirá un alto nivel de desacoplamiento, escalabilidad y mantenibilidad.

El escenario base contempla la organización del sistema por módulos, el modelado general de la solución, la definición de la arquitectura y sus dependencias, así como la estructura necesaria para soportar los diferentes entornos y servicios asociados al funcionamiento del sistema.

Objetivo General

Diseñar una arquitectura de software modular y escalable para el sistema de lavadero de carros, estructurada en microservicios y basada en principios de arquitectura hexagonal.

Objetivos Específicos

1. Modelar la estructura y funcionamiento del sistema mediante diagramas y representación arquitectónica.
2. Definir la arquitectura del sistema, sus módulos, dependencias y flujos principales.
3. Establecer la organización de ambientes y la estructura necesaria para el despliegue y operación del sistema.

Justificación

El funcionamiento de un lavadero de carros depende de una buena organización: saber qué vehículos entran, qué servicios necesitan, cuándo atender a cada cliente y cómo llevar registro de todo. Cuando estos procesos se realizan de manera manual o sin un sistema claro, se generan retrasos, confusiones y mala experiencia para el cliente.

Este proyecto es necesario porque busca darles orden, claridad y eficiencia a estos procesos. Diseñar una arquitectura de software bien estructurada permitirá que el negocio gestione mejor sus servicios, atienda más rápido a las personas y tenga información confiable para tomar decisiones.

La solución beneficia tanto al personal del lavadero que podrá trabajar con menos carga operativa y mayor control como a los clientes, quienes recibirán una atención más fluida y organizada. En general, es un proyecto valioso porque resuelve un problema real y mejora la calidad del servicio ofrecido.

Alcance

Este proyecto abarca exclusivamente el diseño y documentación de la arquitectura del sistema de gestión para un lavadero de carros. Dentro del alcance se desarrollará:

- La definición de los módulos principales: vehículos, usuarios, servicios y reservaciones.
- La elaboración de diagramas y modelos arquitectónicos que muestren cómo se organiza y comunica cada parte del sistema.
- La identificación de dependencias, componentes clave y flujos generales de información.
- La selección y justificación del stack tecnológico adecuado para soportar la arquitectura propuesta.
- La descripción de los ambientes de trabajo necesarios (DEV - QA – PROD - Staging).
- La estructura general para el despliegue del sistema de acuerdo con una arquitectura modular.

Este proyecto se limita al diseño arquitectónico del sistema, sin implementar código ni microservicios reales. Se entrega documentación, modelado y definición técnica de la arquitectura.

MARCO TEÓRICO

1. Arquitectura de Microservicios

La arquitectura de microservicios divide un sistema en servicios pequeños, independientes y orientados a capacidades del negocio. Cada servicio puede desarrollarse, escalarse y desplegarse sin afectar a los demás.

Di Francesco, Lago y Malavolta (2019) destacan beneficios como el despliegue independiente, la escalabilidad y el aislamiento de fallas, aunque también señalan retos como la complejidad de la comunicación distribuida. Barrios Contreras (2018) afirma que esta arquitectura facilita la evolución del sistema y la integración de nuevas funcionalidades.

2. Arquitectura Hexagonal

La arquitectura hexagonal, descrita en AWS Prescriptive Guidance, separa la lógica de negocio de la infraestructura mediante puertos y adaptadores. Esto permite mantener el dominio libre de dependencias externas y facilita el cambio de tecnologías. HappyCoders explica que este patrón mejora la testabilidad, la extensibilidad y el desacoplamiento interno del sistema.

3. Relación entre Microservicios y Hexagonal

Bushong et al. (2021) indican que para que los microservicios evolucionen correctamente deben tener límites claros y separar la lógica de negocio de la infraestructura. La arquitectura hexagonal se ajusta perfectamente a esta necesidad, por lo que es común usarla dentro de cada microservicio para mantener su independencia y claridad estructural.

4. Modelado UML y C4

UML es un lenguaje estándar para describir sistemas mediante diagramas como clases, secuencia, casos de uso y componentes, permitiendo visualizar entidades como usuarios, vehículos o reservaciones.

El modelo C4, propuesto por Simon Brown, organiza la documentación arquitectónica en cuatro niveles: contexto, contenedores, componentes y código, siendo ideal para representar sistemas basados en microservicios.

5. Gestión de Ambientes

Los sistemas modernos requieren ambientes separados como DEV, QA, STG y PROD. Bushong et al. (2021) destacan que estos ambientes son esenciales en arquitecturas distribuidas para garantizar despliegues seguros, pruebas controladas y estabilidad del sistema.

6. Contenedores, Kubernetes y CI/CD

Los contenedores permiten empaquetar aplicaciones de forma portable y consistente, mientras que Kubernetes gestiona su orquestación y escalado. La automatización mediante CI/CD (con herramientas como Jenkins) es clave para mantener ciclos de entrega confiables y rápidos. Malavolta (2019) resalta que este tipo de automatización es crítica en sistemas distribuidos basados en microservicios.

Escenario Base

El lavadero pulido rojo de carros actualmente gestiona sus procesos de manera manual: los vehículos, servicios y reservaciones se registran en cuadernos o por mensajes informales. Esto genera desorden, pérdida de información, dificultad para planificar la atención y una experiencia poco eficiente para los clientes. Con el aumento de la demanda, el negocio necesita una solución tecnológica que organice los datos y automatice los procesos operativos.

A partir de esta necesidad surge el proyecto de diseñar la arquitectura de un sistema de gestión que permita controlar módulos como vehículos, usuarios, servicios y reservaciones. El diseño se basará en microservicios con enfoque de arquitectura hexagonal para asegurar modularidad, escalabilidad y facilidad de mantenimiento.

En este contexto, el escenario base representa la situación actual del negocio y justifica la necesidad de una solución arquitectónica sólida que permita mejorar la organización, optimizar el flujo de trabajo y servir como fundamento para una futura transformación digital del lavadero.

Definición de Etapas

El desarrollo del proyecto del sistema para el Lavadero Pulido Rojo se divide en etapas que responden directamente a los entregables exigidos: modelado, arquitectura, ambientes, stack tecnológico y despliegue. Cada etapa organiza el trabajo de forma clara para cubrir cada componente del documento final.

Etapla 1: Análisis del negocio y definición del alcance

Se recopila información sobre el funcionamiento actual del lavadero, los procesos manuales, los actores y los problemas.

Aquí también se confirman los módulos obligatorios: vehículos, usuarios, servicios y reservaciones.

Esta etapa permite entender qué debe resolver la arquitectura.

Etapla 2: Modelado UML y C4

Se construyen los diagramas que describen el sistema:

- UML: casos de uso, clases, secuencia y componentes.
- C4: contexto, contenedores y componentes.

Estos diagramas permiten visualizar el sistema desde lo general hasta lo detallado.

Etapas 3: Comparativa del stack tecnológico

En esta etapa se evalúan diferentes tecnologías posibles para el proyecto: lenguajes, frameworks, motores de base de datos, plataformas de contenedores, herramientas para documentación, orquestación y despliegue.

Se crea una tabla de ventajas y desventajas para justificar la elección final.

Etapas 4: Definición de la arquitectura y sus dependencias

Se describe la arquitectura basada en microservicios con enfoque hexagonal.

Aquí se definen:

- Módulos del sistema.
- Estructura interna de cada microservicio.
- Puertos y adaptadores.
- Dependencias internas y externas.
- Relaciones entre servicios.
- Tokens, flujos e items del sistema (para procesos relacionados con Camunda/Bizagi).

Esta etapa asegura que el sistema esté correctamente diseñado y desacoplado.

Etapas 5: Modeler (Camunda / Bizagi)

Se modelan los procesos del lavadero:

- Flujo de reservaciones
- Atención del vehículo
- Gestión de servicios
- Registro de cliente o vehículo

El objetivo es representar visualmente cómo se mueve la información y qué decisiones toma el sistema.

Aquí se integran conceptos como flujos, tokens y elementos del proceso.

Etapas 6: Gestión de ambientes

Se definen los ambientes técnicos del proyecto:

- DEV (desarrollo)
- QA (pruebas)
- STAGING (preproducción)

- PROD (producción)

También se organizan los repositorios necesarios:

- repos/docs
- repos/api
- repos/portal
- repos/app
- repos/other_service

Cada ambiente incluye propósito, configuración básica, reglas de despliegue y estándares mínimos.

Etapas 7: Estrategia de despliegue

Se documenta cómo se manejaría el despliegue del sistema en una situación real:

- Contenedores Docker
- Orquestación con Kubernetes
- Pipeline CI/CD (Jenkins o GitHub Actions)
- Versionado
- Integración continua
- Pruebas automatizadas

Esta etapa explica cómo se operaría el sistema una vez implementado.

Etapas 8: Documentación final, presentación y video

Finalmente, se consolidan todas las secciones del proyecto, se organiza el documento final y se prepara:

- Presentación
- Video clip (explicación del proyecto)

Priorización del Proyecto

La priorización del sistema se basa en garantizar que los elementos críticos para el funcionamiento inicial del lavadero queden completamente definidos antes de avanzar hacia componentes complementarios. Se adopta una lógica que prioriza primero los microservicios fundamentales, luego los procesos operativos del negocio y, finalmente, los servicios de soporte.

Categoría (MoSCoW)	Funcionalidad	Descripción / Justificación
M – Must Have	Microservicio de Usuarios	Núcleo para autenticación, registro y control de roles. Sin este módulo no es posible gestionar accesos ni seguridad.
M – Must Have	Microservicio de Vehículos	Permite registrar vehículos y asociarlos a usuarios; esencial para cualquier operación del lavadero.
M – Must Have	Microservicio de Servicios	Define el catálogo de servicios (lavado, encerado, pulido, etc.). Es imprescindible para relacionar usuarios y vehículos con las operaciones.
M – Must Have	Microservicio de Reservaciones	Gestiona disponibilidad, horarios y citas; clave para automatizar el flujo de trabajo del lavadero.
M – Must Have	Definición de arquitectura y dependencias	Incluye diagramas de arquitectura hexagonal por microservicio y el mapa de dependencias entre componentes. Es esencial para establecer las bases del diseño arquitectónico del sistema.
S – Should Have	Gestión de ambientes (DEV, QA, STAGING, PROD)	Necesaria para organizar pruebas, validaciones y despliegues seguros. Importante para control, pero no detiene la definición arquitectura base.
S – Should Have	Estrategia de despliegue (contenedores, Kubernetes, CI/CD)	Garantiza versionado, replicabilidad y automatización del ciclo de entrega, aumentando la eficiencia operativa.
S – Should Have	Modelado BPMN (Camunda / Bizagi)	Permite comprender y documentar los flujos internos de negocio; mejora la claridad arquitectónica y el alineamiento con los procesos del lavadero.
S – Should Have	Definición de tokens, flows e items	Elementos fundamentales para ejecutar procesos BPMN y establecer un flujo operacional completo. Importante, pero no crítico para la arquitectura mínima.
C – Could Have	Notificaciones, reportes y dashboards	Aportan valor agregado y mejoran la experiencia de uso, pero no son indispensables para el funcionamiento inicial.

W – Won't Have	Desarrollo real de microservicios (implementación de código)	No está dentro del alcance del proyecto; el enfoque se mantiene en diseño, modelado y documentación arquitectónica.
-----------------------	--	---

Estimación del Proyecto

La estimación se presenta en días de trabajo basados en un ritmo estándar del análisis, diseño y documentación técnica. En este caso no se incluye desarrollo real porque el proyecto es exclusivamente arquitectónico.

Etapas	Esfuerzo Estimado
Levantamiento de información	1 día
Definición de módulos y alcance	1 día
Diseño arquitectónico (microservicios + hexagonal + dependencias)	2 – 3 días
Modelado UML y C4	1 – 2 días
Modelado BPMN (Camunda o Bizagi)	2 días
Comparativa de stack tecnológico	1 día
Gestión de ambientes y repositorios	1 – 2 días
Estrategia de despliegue (contenedores, Kubernetes, CI/CD)	2 – 3 días
Documentación final, ajustes y presentación	1 – 2 días

El tiempo estimado de este proyecto es de 12 a 17 días de trabajo continuo, pero también dependerá del detalle y complejidad de cada diagrama y sección arquitectónica.

Costos del Proyecto

Para estimar el coste de este proyecto se tomará como referencia la tarifa promedio de un desarrollador de nivel intermedio en Colombia, que oscila entre \$80.000 y \$120.000 COP por hora para trabajos técnicos que incluyen análisis, diseño arquitectónico, modelado y documentación.

Basándonos en un esfuerzo aproximado de **288 y 408 horas** dedicadas al proyecto completo (incluyendo investigación, diseño, diagramación y documentación), el cálculo sería:

Costo por desarrollador

- 288 y 408 horas × \$120.000 COP/hora

Rango estimado:

- \$ 34,560,000
- \$48,960,000

Costo total del proyecto con dos integrantes

Dado que tu equipo es de dos personas, el coste total estimado sería:

- \$69,120,000 – \$97,920,000 COP

Este valor se justifica por:

- La cantidad de diagramas (UML, C4, BPMN).
- El nivel de detalle requerido en la arquitectura hexagonal.
- La documentación formal.
- El análisis comparativo del stack tecnológico.
- La definición de ambientes, despliegue y dependencias.

Comparativa de stack tecnológico

La selección del stack tecnológico es una parte fundamental del diseño arquitectónico, ya que determina las herramientas, lenguajes, frameworks y plataformas que soportarán el desarrollo, despliegue, modelado y operación del sistema del Lavadero Pulido Rojo.

Para garantizar la elección más adecuada, se realizó una comparación entre las tecnologías utilizadas por el equipo (Java, Spring Boot, Docker, Kubernetes, Jenkins, Bizagi, VS Code y GitHub) y otras alternativas comúnmente empleadas en proyectos similares.

El objetivo de esta comparativa es justificar técnicamente por qué cada herramienta seleccionada es la más conveniente para el contexto del proyecto, teniendo en cuenta factores como rendimiento, curva de aprendizaje, escalabilidad, mantenibilidad y alineación con arquitecturas basadas en microservicios.

Lenguaje + Framework

Tecnología	Ventajas	Desventajas
Java + Spring Boot (Seleccionado)	• Muy robusto para microservicios. • Excelente integración con Docker, Kubernetes y CI/CD. • Amplio soporte empresarial.	• Curva de aprendizaje más alta. • Requiere más recursos que Node.js.

	• Ecosistema maduro (Spring Cloud).	
Node.js + Express	• Fácil aprendizaje. • Rápido para prototipos. • Muy buen ecosistema NPM.	• Menor robustez para proyectos grandes. • Dificultad en manejo de procesos pesados.
Python + Django	• Framework muy estructurado. • Admin panel integrado.	• Más orientado a monolitos. • Menos adecuado para microservicios complejos.

Base de Datos

Tecnología	Ventajas	Desventajas
MySQL (Seleccionado)	• Ligera, rápida y fácil de usar. • Ideal para sistemas transaccionales como un lavadero. • Integración nativa con Spring Boot.	• Menos potente para análisis complejos comparado con PostgreSQL.
PostgreSQL	• Muy robusta y con más funciones avanzadas.	• Mayor complejidad y consumo de recursos.

Contenedores y Orquestación

Tecnología	Ventajas	Desventajas
Docker + Kubernetes (Seleccionado)	• Estándar de la industria. • Escalabilidad horizontal garantizada. • Aislamiento perfecto entre microservicios.	• Complejidad de aprendizaje. • Requiere buena gestión de recursos.
Docker Compose	• Simple para desarrollo local.	• No sirve para producción escalada.

CI/CD

Tecnología	Ventajas	Desventajas
------------	----------	-------------

Jenkins (Seleccionado)	• Extremadamente configurable. • Amplio soporte de plugins.	• Interfaz menos amigable. • Curva de aprendizaje.
GitHub Actions	• Fácil de integrar con repos GitHub. • Menos compleja de configurar.	• Menos flexible para pipelines avanzados.

Modelado y Diseño

Tecnología	Ventajas	Desventajas
Bizagi Modeler (Seleccionado)	• Estándar académico para BPMN. • Fácil uso y diagramas claros.	• No permite simulación avanzada sin versión de pago.
Camunda Modeler	• Más técnico y orientado a desarrolladores.	• Mayor complejidad para principiantes.

Gestión del Proyecto

Tecnología	Ventajas	Desventajas
Trello (Seleccionado)	• Simple y visual. • Ideal para backlog y Kanban.	• Menos completo que Jira.
Jira	• Super completo para equipos grandes.	• Excesivo para proyectos académicos.

Control de versiones

Tecnología	Ventajas	Desventajas
GitHub (Seleccionado)	• Sencillo, rápido y estándar académico.	• Menos opciones avanzadas que GitLab.
GitLab	• CI/CD integrado muy completo.	• Curva de aprendizaje mayor.

Teniendo en cuenta la comparación de herramientas tecnológicas que se basó en su compatibilidad con microservicios, facilidad de integración, eficiencia operativa,

escalabilidad y soporte técnico, esta siguiente tabla resume las tecnologías utilizadas y las razones por las cuales fueron seleccionadas luego de haber hecho las comparaciones.

Tecnología Seleccionada	Justificación de Selección
Java + Spring Boot	Ofrece una arquitectura robusta y estable para microservicios, con un ecosistema maduro (Spring Cloud), excelente integración con Docker/Kubernetes y alto rendimiento en entornos empresariales.
MySQL	Base de datos ligera, rápida y altamente confiable para sistemas transaccionales como un lavadero. Se integra muy bien con Spring Boot y es fácil de administrar.
Docker + Kubernetes	Garantizan despliegues aislados, escalables y estandarizados. Kubernetes permite gestionar múltiples microservicios con balanceo, autoscaling y alta disponibilidad.
Jenkins	Herramienta CI/CD extremadamente flexible y configurable, ideal para pipelines complejos e integraciones con contenedores, orquestación y repositorios.
Bizagi Modeler	Herramienta clara, intuitiva y académica para modelar procesos BPMN, facilitando la documentación del flujo operativo del lavadero.
Trello	Simple, visual y efectivo para gestionar backlog, priorización y Kanban del proyecto sin sobrecargar al equipo.
GitHub	Plataforma estándar para repositorios, colaboración y control de versiones; fácil de usar y con integración directa con Jenkins y herramientas del stack.

- **Conclusión 1**

El diseño arquitectónico realizado para el Lavadero Pulido Rojo demuestra que una buena planificación tecnológica puede transformar procesos que hoy son manuales y desordenados en un sistema organizado, claro y eficiente. Al estructurar la solución con microservicios y una arquitectura hexagonal, se garantiza que el sistema pueda crecer, adaptarse y mantenerse sin depender de tecnologías específicas, asegurando su vigencia a largo plazo.

-

A través de las etapas de modelado UML, C4, BPMN y la definición de ambientes, se logró construir una visión completa y coherente de cómo debe funcionar el sistema, desde sus procesos internos hasta su despliegue en entornos reales. Esto no solo aporta claridad técnica, sino que también ofrece un camino seguro para una futura implementación, evitando improvisaciones y disminuyendo riesgos.

-

Finalmente, la comparación y selección del stack tecnológico reafirma que cada

herramienta elegida responde a necesidades reales del proyecto: robustez, escalabilidad, facilidad de integración y soporte. Gracias a estas decisiones, el proyecto se consolida como una propuesta viable, moderna y totalmente alineada con buenas prácticas de arquitectura de software, aportando un valor tangible al negocio y a sus usuarios.

Referencias

Di Francesco, P., Lago, P., & Malavolta, I. (2019). Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*, 150, 77-97.

https://www.ivanomalavolta.com/files/papers/JSS_MSA_2019.pdf?utm_source

Bushong, V., Abdelfattah, A. S., Maruf, A. A., Das, D., Lehman, A., Jaroszewski, E., ... Bures, M. (2021). On Microservice Analysis and Architecture Evolution: A Systematic Mapping Study. *Applied Sciences*, 11 (17), 7856. https://www.mdpi.com/2076-3417/11/17/7856?utm_source

AWS Prescriptive Guidance. Hexagonal architecture pattern.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html?utm_source

HappyCoders. Hexagonal Architecture – What Is It? Why Use It?

https://www.happycoders.eu/software-craftsmanship/hexagonal-architecture/?utm_source

Barrios Contreras, D. A. (2018). Arquitectura de Microservicios. *Tecnología Investigación y Academia*, 6(1), 36–46. <https://revistas.udistrital.edu.co/index.php/tia/article/view/9687>